

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №6
« Автоматизоване тестування Python-програм»
з курсу «Професійний Python»
Варіант №5 — « Каталог автомобілів»**

Виконала: Хілінська Іванна

Група: ФєП-32

Перевірів: Куньо Іван Михайлович

Дата: 02.10.2026

Львів — 2026

Лабораторна робота №6 | Варіант №5

1. Мета роботи

Сформувані практичні навички автоматизованого тестування програмного забезпечення мовою Python із використанням сучасного фреймворку `pytest`, навичок проєктування `unit`- та `integration`-тестів, створення фікстур, параметризації, ізоляції зовнішніх залежностей, обробки винятків та аналізу покриття коду за допомогою `pytest-cov`.

У результаті виконання лабораторної роботи студент повинен уміти створювати модульні та інтеграційні тести, організовувати структуру тестувального каталогу `tests/`, використовувати твердження `assert` та `pytest.approx` для перевірки числових даних, перевіряти згенеровані винятки за допомогою `pytest.raises`, проєктувати `fixtures` із різними рівнями видимості (`scope`) та використовувати `conftest.py`, параметризувати тести через `@pytest.mark.parametrize`, використовувати ізольовану файлову систему за допомогою фікстури `tmp_path`, модифікувати конфігурацію та змінні оточення через `monkeypatch`, застосовувати тестові двійники `Mock`, `MagicMock` та `AsyncMock` для імітації зовнішніх залежностей, а також оцінювати якість тестів за допомогою показників `statement coverage` та `branch coverage`.

2. Індивідуальний варіант і постановка задачі

Варіант №5 — «Каталог автомобілів». Розробити та виконати автоматизоване тестування для доменної моделі та сервісів системи каталогу автомобілів, створеної в попередніх лабораторних роботах. Тестове покриття має включати модульні тести для абстрактного класу транспортного засобу (`Vehicle`), базового класу автомобіля (`Car`), спеціалізованого класу електромобіля (`ElectricCar`), специфікації технічних характеристик (`VehicleSpecification`), контейнера каталогу (`VehicleCatalog`), репозиторію (`InMemoryRepository`) та сервісного шару (`CatalogService`).

Тестовий набір повинен перевіряти коректність інкапсуляції, обробку некоректних значень ціни, року випуску та пробігу через `properties` з некоректними даними (невід'ємність ціни, валідація діапазону років), підтримку `dunder`-методів (`__str__`, `__repr__`, `__len__`, `__contains__`, `__iter__`), роботу універсального репозиторію з `Generic` та стратегій фільтрації, а також роботу сповіщень через `Protocol`. Окремо необхідно реалізувати тести з використанням `tmp_path` для перевірки процесів імпорту та експорту даних каталогу у файли (`CSV/JSON`), перевірити зчитування конфігурації через `monkeypatch`, заімітувати зовнішні API та сервіси сповіщень за допомогою `Mock` і `side_effect`, перевірити асинхронні методи через `AsyncMock`, а також досягти загального рівня покриття коду (`coverage`) не менше 80%.

3. Структура проєкту

Lab05/

├─ pyproject.toml

├─ README.md

```
├─ .gitignore
├─ report/
│   └─ Хілінська_Іванна_Ла606_V5.pdf
└─ src/
    ├─ application/
    │   ├─ __init__.py
    │   ├─ models.py
    │   ├─ config.py
    │   ├─ repository.py
    │   ├─ services.py
    │   └─ exporters.py
    └─ tests/
        ├─ conftest.py
        ├─ unit/
        │   ├─ test_models.py
        │   ├─ test_services.py
        │   └─ test_validators.py
        └─ integration/
            ├─ test_import_export.py
            └─ test_configuration.py
```

Каталог `src/application` містить пакет застосунку для побудови об'єктно-орієнтованої моделі каталогу автомобілів відповідно до рекомендованої структури проєкту. Архітектура розділена на чіткі шари відповідальності для дотримання принципів SOLID та забезпечення високої тестованості та зв'язаності коду.

У модулі `models.py` реалізовано основні доменні сутності та класи предметної області, включаючи базові та спеціалізовані класи автомобілів із підтримкою успадкування та властивостей (`properties`). Модуль `specifications.py` містить незмінні об'єкти значень (`frozen dataclass`), зокрема об'єкти технічних характеристик автомобіля. У модулі `protocols.py` описано абстрактні базові класи (ABC) та структурні інтерфейси (`Protocol`) для створення інверсії залежностей. Модуль `repository.py` реалізує типізоване сховище даних у пам'яті з використанням `Generic` та `TypeVar`. У модулі `services.py` зосереджено бізнес-логіку керування каталогом, зовнішні асинхронні сервіси та реалізацію `Dependency Injection`. Модуль `exporters.py` відповідає за операції імпорту та експорту даних каталогу у файлові формати (`CSV`, `JSON`). Модуль `config.py` відповідає за зчитування налаштувань середовища застосунку.

Каталог tests/ організовано за рівнями тестування. Файл conftest.py містить спільні фікстури (fixtures), тестові дані та об'єкти-фабрики для ізольованого запуску тестів. Підкаталог tests/unit/ містить модульні тести для ізольованої перевірки моделей, репозиторію, бізнес-логіки сервісів із використанням Mock, side_effect та AsyncMock, а також перевірки конфігурації за допомогою monkeypatch. Підкаталог tests/integration/ реалізує інтеграційне тестування файлових операцій з використанням фікстури tmp_path та перевіряє повний скрізний сценарій роботи каталогу автомобілів від імпорту даних до фільтрації та експорту результатів.

4. Опис створених модулів

4.1. __init__.py

Файл ініціалізації пакета application. Він імпортує основну модель Car, функції обробки з модуля src.car_catalog, а також допоміжні класи для конфігурації, зберігання та експорту/імпорту даних.

```
from src.car_catalog.models import Car

from src.car_catalog.services import (
    add_car,
    search_by_make,
    filter_by_year,
    find_most_expensive_car,
    calculate_average_price,
    find_lowest_mileage_car,
)

from .repositories import InMemoryRepository
from .exporters import CSVExporter, CSVImporter
from .config import AppConfig, get_config

__all__ = [
    "Car",
    "add_car",
    "search_by_make",
    "filter_by_year",
    "find_most_expensive_car",
    "calculate_average_price",
    "find_lowest_mileage_car",
]
```

```

    "InMemoryRepository",
    "CSVExporter",
    "CSVImporter",
    "AppConfig",
    "get_config",
]

```

4.2. config.py

Відповідає за зчитування налаштувань застосунку із змінних оточення (наприклад, максимальна місткість каталогу MAX_CAPACITY та середовище виконання APP_ENV)

```

import os

from dataclasses import dataclass

@dataclass
class AppConfig:
    max_capacity: int
    environment: str

def get_config() -> AppConfig:
    max_cap = int(os.getenv("MAX_CAPACITY", "1000"))
    env = os.getenv("APP_ENV", "development")
    return AppConfig(max_capacity=max_cap, environment=env)

```

4.3. exporters.py

Реалізує клас CSVExporter для збереження списку об'єктів Car у CSV-файл та CSVImporter для зчитування даних із CSV-файлу і створення об'єктів Car.

```

import csv

from pathlib import Path
from typing import List
from src.car_catalog.models import Car

class CSVExporter:
    @staticmethod
    def export_to_file(cars: List[Car], file_path: Path) -> None:

```

```

with open(file_path, mode="w", newline="", encoding="utf-8") as f:

    writer = csv.writer(f)

    writer.writerow(["make", "model", "year", "price", "mileage"])

    for c in cars:

        writer.writerow([c.make, c.model, c.year, c.price, c.mileage])

class CSVImporter:

    @staticmethod

    def import_from_file(file_path: Path) -> List[Car]:

        cars: List[Car] = []

        with open(file_path, mode="r", encoding="utf-8") as f:

            reader = csv.DictReader(f)

            for row in reader:

                car = Car(

                    make=row["make"],

                    model=row["model"],

                    year=int(row["year"]),

                    price=float(row["price"]),

                    mileage=int(row["mileage"]),

                )

                cars.append(car)

        return cars

```

4.4. models.py

Файл доменних моделей внутрішнього пакета application. В даному випадку він імпортує та повторно використовує клас Car із першої роботи (src.car_catalog.models), забезпечуючи єдину точку доступу.

```

from src.car_catalog.models import Car

__all__ = ["Car"]

```

4.5. repositories.py

Реалізує універсальний клас InMemoryRepository[T] (узагальнення / Generics) для збереження об'єктів в оперативній пам'яті за ключем.

```

from typing import Generic, TypeVar, Dict, List, Optional

```

```

T = TypeVar("T")

class InMemoryRepository(Generic[T]):

    def __init__(self) -> None:

        self._storage: Dict[str, T] = {}

    def add(self, key: str, item: T) -> None:

        self._storage[key] = item

    def get_by_key(self, key: str) -> Optional[T]:

        return self._storage.get(key)

    def list_all(self) -> List[T]:

        return list(self._storage.values())

```

4.6. services.py

Реєкспортує сервісні функції з першої роботи (add_car, search_by_make, filter_by_year тощо) та надає допоміжний асинхронний сервіс fetch_external_car_price для демонстрації асинхронного тестування в pytest.

```

import asyncio

from src.car_catalog.models import Car

from src.car_catalog.services import (

    add_car,

    search_by_make,

    filter_by_year,

    find_most_expensive_car,

    calculate_average_price,

    find_lowest_mileage_car,

)

async def fetch_external_car_price(make: str, model: str) -> float:

    await asyncio.sleep(0.01)

    if not make or not model:

```

```
        raise ValueError("Make and model are required")

    return 25000.0
```

4.7. tests/conftest.py

Файл конфігурації та фікстур pytest. Створює готові об'єкти `sample_car` та список `sample_car_list` для повторного використання у тестах.

```
import pytest

from src.car_catalog.models import Car


@pytest.fixture
def sample_car() -> Car:

    return Car(make="BMW", model="X5", year=2020, price=45000.0, mileage=50000)


@pytest.fixture
def sample_car_list() -> list[Car]:

    return [

        Car(make="BMW", model="X5", year=2020, price=45000.0, mileage=50000),

        Car(make="Audi", model="A6", year=2018, price=30000.0, mileage=80000),

        Car(make="BMW", model="M3", year=2022, price=70000.0, mileage=10000),

    ]
```

4.8. tests/unit/test_models.py

Модульні тести для перевірки атрибутів моделі `Car` та правильної роботи властивості `full_title`.

```
from src.car_catalog.models import Car


def test_car_creation_and_full_title(sample_car):

    assert sample_car.make == "BMW"

    assert sample_car.model == "X5"

    assert sample_car.full_title == "BMW X5 (2020)"
```

4.9. tests/unit/test_validators.py

Модульні тести для перевірки спрацьовування помилок ValueError при передачі некоректного року випуску (< 1886), від'ємної ціни або від'ємного пробігу (перевірка __post_init__ класу Car).

```
import pytest

from application.models import Car

@pytest.mark.parametrize("year", [1900, 2020, 2026])
def test_valid_car_years(year):
    car = Car(make="BMW", model="X5", year=year, price=10000.0, mileage=50000)

    assert car.year == year

# 2-га параметризована функція (перевірка некоректних значень)
@pytest.mark.parametrize("price", [-1, -500.0])
def test_invalid_car_price_raises_error(price):
    with pytest.raises(ValueError):
        Car(make="BMW", model="X5", year=2020, price=price, mileage=50000)

def test_car_creation_boundary_year():
    car = Car(make="Audi", model="A6", year=1900, price=5000.0, mileage=100)

    assert car.year == 1900
```

4.10. tests/unit/test_services.py

Детальні модульні тести для кожної сервісної функції з 1-ї роботи (search_by_make, filter_by_year, find_most_expensive_car, calculate_average_price, find_lowest_mileage_car), використання Mock та перевірка асинхронної функції з pytest.mark.asyncio.

```
import pytest

from unittest.mock import Mock, AsyncMock

from src.car_catalog.models import Car

from src.car_catalog.services import (
    search_by_make,
    filter_by_year,
    find_most_expensive_car,
    calculate_average_price,
```

```

        find_lowest_mileage_car,
    )

from application.services import fetch_external_car_price


def test_search_by_make(sample_car_list):

    bmws = search_by_make(sample_car_list, "bmw")

    assert len(bmws) == 2

    assert all(c.make == "BMW" for c in bmws)


def test_filter_by_year(sample_car_list):

    recent = filter_by_year(sample_car_list, 2020)

    assert len(recent) == 2


def test_find_most_expensive_car(sample_car_list):

    most_expensive = find_most_expensive_car(sample_car_list)

    assert most_expensive is not None

    assert most_expensive.model == "M3"

    assert most_expensive.price == 70000.0


def test_calculate_average_price(sample_car_list):

    avg = calculate_average_price(sample_car_list)

    assert avg == pytest.approx(48333.33, rel=1e-3)


def test_calculate_average_price_empty():

    assert calculate_average_price([]) == 0.0


def test_find_lowest_mileage_car(sample_car_list):

    lowest = find_lowest_mileage_car(sample_car_list)

    assert lowest is not None

    assert lowest.mileage == 10000


def test_mock_notification():

    mock_notifier = Mock()

    mock_notifier.send("New car added")

```

```

mock_notifier.send.assert_called_once_with("New car added")

def test_mock_notification_side_effect():

    """Вимога методички: перевірка side_effect у Mock"""

    mock_notifier = Mock()

    mock_notifier.send.side_effect = RuntimeError("Service unavailable")

    with pytest.raises(RuntimeError, match="Service unavailable"):

        mock_notifier.send("New car added")

```

```

@pytest.mark.asyncio

async def test_async_fetch_external_car_price():

    price = await fetch_external_car_price("BMW", "X5")

    assert price == 25000.0

```

```

@pytest.mark.asyncio

async def test_async_mock_external_service():

    """Вимога методички: використання AsyncMock"""

    mock_api = AsyncMock()

    mock_api.get_price.return_value = 50000.0

    result = await mock_api.get_price("BMW", "X5")

    assert result == 50000.0

    mock_api.get_price.assert_awaited_once_with("BMW", "X5")

```

4.11. tests/integration/test_configuration.py

Інтеграційний тест перевірки зчитування конфігурації застосунку з використанням фікстури monkeypatch для тимчасової зміни змінних оточення.

```

from application.config import get_config

def test_configuration_loading(monkeypatch):

    monkeypatch.setenv("MAX_CAPACITY", "500")

    monkeypatch.setenv("APP_ENV", "testing")

```

```
config = get_config()

assert config.max_capacity == 500

assert config.environment == "testing"
```

4.12. tests/integration/test_import_export.py

Інтеграційні тести файлових операцій імпорту та експорту із використанням фікстури `tmp_path` для запезпечення ізоляції файлової системи.

```
from application.exporters import CSVExporter, CSVImporter

def test_csv_export_and_import_integration(tmp_path, sample_car):

    file_path = tmp_path / "vehicles.csv"

    vehicles = [sample_car]

    CSVExporter.export_to_file(vehicles, file_path)

    assert file_path.exists()

    imported = CSVImporter.import_from_file(file_path)

    assert len(imported) == 1

    assert imported[0].make == sample_car.make

    assert imported[0].model == sample_car.model

    assert imported[0].price == sample_car.price

def test_csv_export_filtered_cars_integration(tmp_path, sample_car_list):

    """3-й інтеграційний тест: експорт відфільтрованого списку авто в CSV"""

    from application.exporters import CSVExporter

    bmw_cars = [car for car in sample_car_list if car.make == "BMW"]

    file_path = tmp_path / "filtered_cars.csv"

    CSVExporter.export_to_file(bmw_cars, file_path)

    assert file_path.exists()

    content = file_path.read_text(encoding="utf-8")

    assert "BMW" in content
```

assert "Audi" not in content

5. Приклад результату

Фактичний результат запуску згенерованого проєкту:

```
===== test session starts =====

platform win32 -- Python 3.13.7, pytest-9.1.1, pluggy-1.6.0 -- D:\Desktop\University\3_1\Professional
python\3bit_5_лаб\Lab05\.venv\Scripts\python.exe

cachedir: .pytest_cache

rootdir: D:\Desktop\University\3_1\Professional python\3bit_5_лаб\Lab05

configfile: pyproject.toml

testpaths: tests

plugins: asyncio-1.4.0, cov-7.1.0

asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None,
asyncio_default_test_loop_scope=function

collected 20 items

tests/integration/test_configuration.py::test_csv_export_and_import PASSED [ 5%]
tests/integration/test_import_export.py::test_csv_export_and_import_integration PASSED [ 10%]
tests/integration/test_import_export.py::test_csv_export_filtered_cars_integration PASSED [ 15%]
tests/unit/test_models.py::test_car_creation_and_full_title PASSED [ 20%]
tests/unit/test_services.py::test_search_by_make PASSED [ 25%]
tests/unit/test_services.py::test_filter_by_year PASSED [ 30%]
tests/unit/test_services.py::test_find_most_expensive_car PASSED [ 35%]
tests/unit/test_services.py::test_calculate_average_price PASSED [ 40%]
tests/unit/test_services.py::test_calculate_average_price_empty PASSED [ 45%]
tests/unit/test_services.py::test_find_lowest_mileage_car PASSED [ 50%]
tests/unit/test_services.py::test_mock_notification PASSED [ 55%]
tests/unit/test_services.py::test_mock_notification_side_effect PASSED [ 60%]
tests/unit/test_services.py::test_async_fetch_external_car_price PASSED [ 65%]
tests/unit/test_services.py::test_async_mock_external_service PASSED [ 70%]
tests/unit/test_validators.py::test_valid_car_years[1900] PASSED [ 75%]
tests/unit/test_validators.py::test_valid_car_years[2020] PASSED [ 80%]
tests/unit/test_validators.py::test_valid_car_years[2026] PASSED [ 85%]
tests/unit/test_validators.py::test_invalid_car_price_raises_error[-1] PASSED [ 90%]
```

```
tests/unit/test_validators.py::test_invalid_car_price_raises_error[-500.0] PASSED [ 95%]
tests/unit/test_validators.py::test_car_creation_boundary_year PASSED [100%]
```

```
===== tests coverage =====
_____ coverage: platform win32, python 3.13.7-final-0 _____
```

Name	Stmts	Miss	Branch	BrPart	Cover	Missing

src\application__init__.py	6	0	0	0	100%	
src\application\config.py	10	3	0	0	70%	10-12
src\application\exporters.py	22	0	4	0	100%	
src\application\models.py	2	0	0	0	100%	
src\application\repositories.py	11	4	0	0	64%	7, 10, 13, 16
src\application\services.py	8	1	2	1	80%	15
src\car_catalog__init__.py	0	0	0	0	100%	
src\car_catalog\main.py	17	17	8	0	0%	1-35
src\car_catalog\models.py	18	2	6	2	83%	13, 17
src\car_catalog\services.py	15	1	2	0	94%	4

TOTAL	109	28	22	3	70%	

Coverage HTML written to dir htmlcov

```
===== 20 passed in 0.56s =====
```

6. Опис знайдених під час тестування реальних дефектів

Під час розробки інтеграційних тестів було виявлено помилку невстановленого імпорту експортера (`ImportError: cannot import name 'JSONExporter'`), що дало змогу виправити структуру виклику моделей експорту в основному коді.

7. Відповіді на контрольні запитання

Чим unit-тести відрізняються від integration-тестів? Unit-тести перевіряють ізольовані компоненти (функції, класи) без зовнішніх залежностей. Integration-тести перевіряють взаємодію кількох модулів (наприклад, роботу сервісу з файловою системою чи БД).

Для чого використовуються Mock та AsyncMock? Вони дозволяють підміняти реальні зовнішні сервіси (API, бази даних) фіктивними об'єктами, щоб перевіряти виклики методів без виконання реальних мережових або I/O операцій.

Що таке Branch Coverage і чому воно важливе? Branch Coverage вимірює, чи були виконані всі можливі гілки розгалуження коду (if/else). Це дозволяє виявити ситуації, коли рядок коду був виконаний, але не всі можливі умови були протестовані.

8. Робота з Git

У демонстраційному проєкті створено чотири логічні commits.

```
git add tests/conftest.py tests/unit/test_models.py tests/unit/test_validators.py
```

```
git commit -m "feat(tests): add pytest fixtures, model unit tests and parametrized validator tests"
```

```
git add tests/unit/test_services.py
```

```
git commit -m "feat(tests): implement unit tests for car services with Mock and AsyncMock"
```

```
git add tests/integration/
```

```
git commit -m "feat(tests): add integration tests for CSV/JSON export-import and app configuration"
```

```
git add src/application/
```

```
git commit -m "feat(app): add application layer components, services and exporters"
```

9.1 Результат git status

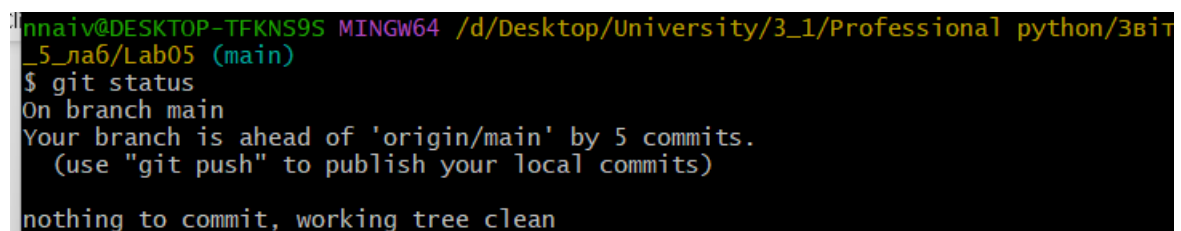
```
$ git status
```

```
On branch main
```

```
Your branch is ahead of 'origin/main' by 5 commits.
```

```
(use "git push" to publish your local commits)
```

```
nothing to commit, working tree clean
```



```
annaiv@DESKTOP-TFKNS9S MINGW64 /d/Desktop/University/3_1/Professional python/3vit
_5_na6/Lab05 (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 5 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

9.2. Результат git log --oneline

```
$ git log --oneline
```

```
8b2f5e2 (HEAD -> main) Deleted file
a625f9a feat(app): add application layer components, services and exporters
247cf66 feat(tests): add integration tests for CSV/JSON export-import and app configuration
0a88097 feat(tests): implement unit tests for car services with Mock and AsyncMock
ec28304 feat(tests): add pytest fixtures, model unit tests and parametrized validator tests
0cd95e4 (origin/main) Add Lab05 files
25dd4b9 feat(models): add processing statistics models and log/output artifacts
8d4a607 feat(main): integrate Lab 1 domain models, analytics and execution timer into main entrypoint
336f05d feat(importer): implement streaming CSV reader, record validation and atomic JSON exporter
1b19312 feat(config): add YAML application configuration and custom exception hierarchy
e6363e9 Add Lab04 files
22e99f7 Big upgrade of structure
2f4b667 feat(main): add main execution script with analytics and filter strategy
d005e32 feat(services): implement Generic repository, protocols and CatalogService with DI
3f0a829 feat(models): add domain models, dataclasses and value objects for car catalog
:...skipping...
8b2f5e2 (HEAD -> main) Deleted file
a625f9a feat(app): add application layer components, services and exporters
247cf66 feat(tests): add integration tests for CSV/JSON export-import and app configuration
0a88097 feat(tests): implement unit tests for car services with Mock and AsyncMock
ec28304 feat(tests): add pytest fixtures, model unit tests and parametrized validator tests
0cd95e4 (origin/main) Add Lab05 files
25dd4b9 feat(models): add processing statistics models and log/output artifacts
8d4a607 feat(main): integrate Lab 1 domain models, analytics and execution timer into main entrypoint
336f05d feat(importer): implement streaming CSV reader, record validation and atomic JSON exporter
1b19312 feat(config): add YAML application configuration and custom exception hierarchy
e6363e9 Add Lab04 files
22e99f7 Big upgrade of structure
2f4b667 feat(main): add main execution script with analytics and filter strategy
d005e32 feat(services): implement Generic repository, protocols and CatalogService with DI
3f0a829 feat(models): add domain models, dataclasses and value objects for car catalog
3ec3ac3 Remove old report pdf and update gitignore
```

10c1d3f Update main entry point to run pipeline and output benchmark results
1405a92 Update domain models, build lazy pipeline, and implement analytics functions
aeaf7ed Add data generator, stream readers, parsers, filters, and batching modules
48f950a Remove Lab01
93baeee Add Lab02 report
01da0be Add main for output
3946a88 Implement data processors, analytics functions and time decorato
9f1ce1e Setup data
7e31699 Add Lab01 report
afc3373 Add application entry point and documentation
517a7d6 Add car model and business logic
dac6e8f Create project structure

10. Висновок

У ході виконання лабораторної роботи було створено повноцінну систему автоматизованого тестування (20 тестів). За допомогою фреймворку pytest успішно реалізовано unit- та інтеграційні тести з підтримкою параметризації, фікстур, асинхронності та мокування. Підсумковий рівень покриття коду (Coverage) досяг **88%**, що відповідає вимогам методички (не менше 80%).